

Clean Code & Refactoring

Nommage · Fonctions · Loi de Déméter · POLA · Dette technique · Code smells · Méthode Mikado

Pourquoi les patterns et SOLID ne suffisent pas

Le constat de Martin (Clean Code, 2008)

- La proportion de temps passé à lire du code vs à en écrire dépasse 10 pour 1
- Un code peut respecter SOLID et rester illisible
- La lisibilité est un levier de productivité direct

Ce que les patterns ne disent pas

- Comment nommer une variable ou une méthode
- Comment calibrer la longueur d'une fonction
- Comment éviter les effets de bord cachés
- Comment gérer la dette technique au quotidien

L'objectif de cette matinée

- Compléter les acquis en patterns par les pratiques qui agissent au niveau le plus fin : le code lui-même
- Du nommage expressif jusqu'aux techniques de refactoring structuré sur du code legacy

Session 1

Clean Code & Code Smells avancés

Nommage · Fonctions · LoD · POLA · Dette · Smells

Nommage expressif

Principes fondamentaux

- Révéler l'intention : pourquoi existe-t-il, que fait-il, comment s'utilise-t-il
- Eviter la désinformation : ne pas évoquer une structure différente de la réalité
- Noms prononçables : si on ne peut pas le dire à voix haute, la communication est difficile
- Noms recherchables : MAX_CLASSES préférable au chiffre 7
- Un mot par concept : fetch / get / retrieve ne doivent pas coexister
- Nommer selon le domaine métier (Ubiquitous Language)

Conventions par type d'entité

- Classe : substantif, PascalCase — InvoiceGenerator
- Méthode : verbe + complément — findByEmail()
- Booléen : préfixe is / has / can — isActive, hasPermission
- Constante : SCREAMING_SNAKE_CASE — MAX_RETRY_ATTEMPTS
- Paramètre : descriptif, non abrégé — recipientEmail, pas e

Fonctions courtes & abstraction par niveau

Règle de l'abstraction uniforme (SLA)

- Une fonction ne doit opérer qu'à un seul niveau d'abstraction
- Tous ses appels internes se situent au même niveau
- Mélanger SQL brut et règles métier dans la même fonction rend le code illisible

Nombre de paramètres

- Monadique (1 paramètre) : idéal
- Dyadique (2) : acceptable
- Au-delà de 3 : envisager un objet de paramètres
- Eviter les paramètres booléens (signalent une double responsabilité)

Eviter les effets de bord cachés

- Une fonction nommée checkPassword ne devrait pas initialiser une session ou envoyer un email
- Les effets de bord non annoncés sont la première source de bugs difficiles à tracer
- Règle : les fonctions getter / checker doivent être pures ; les effets sont dans des fonctions explicitement nommées

Loi de Déméter & Principe du moindre étonnement

Loi de Déméter (LoD) — 1987

- « Ne parle qu'à tes amis directs »
- Un objet ne doit interagir qu'avec : ses propres attributs, les paramètres reçus, les objets qu'il crée
- Eviter le train wreck :
`order.getCustomer().getLoyaltyAccount().getRate()`
- Exception documentée : les DSL fluents (query builder, builder pattern)
- Correction : déléguer à l'objet le soin d'exposer ce qui est nécessaire

Principe du moindre étonnement (POLA)

- Un composant doit se comporter comme ses utilisateurs s'y attendent
- Violations typiques :
 - Un getter qui produit un effet de bord réseau
 - Un constructeur qui ouvre une connexion BDD
 - Une méthode `save()` qui retourne parfois null et parfois lève une exception
 - Une méthode `isEmpty()` qui modifie l'état interne

Dette technique

Métaphore de Ward Cunningham (1992) · Quadrant de Fowler (2009)

Dette technique — Définition & quadrant de Fowler

Définition (Ward Cunningham, 1992)

- Prendre un raccourci dans le code aujourd'hui = contracter une dette dont les intérêts se paient sous forme de temps de développement supplémentaire dans le futur

Délibérée & Imprudente

- « On n'a pas le temps de bien faire »
- Mauvaise dette — à éviter

Délibérée & Prudente

- « On livre maintenant et on refactorise ensuite »
- Seule forme acceptable — décision consciente et documentée

Inconsciente & Imprudente

- « C'est quoi un pattern ? »
- Ignorance technique — à traiter par la formation

Inconsciente & Prudente

- « On savait qu'il fallait faire autrement, mais on ne savait pas comment »
- Se découvre avec l'expérience

Mesure de la dette & règle du boy scout

Métriques de code statique

- Complexité cyclomatique — seuil d'alerte : > 10
- Longueur de fonction — seuil : > 20–30 lignes
- Profondeur d'imbrication (if/for/while) — seuil : > 3
- Couplage afférent (Ca) : classes qui dépendent de X — seuil : > 20
- Couplage efférent (Ce) : classes dont X dépend — seuil : > 10
- LCOM — manque de cohésion des méthodes — seuil : > 0.8

Outils

- Python : radon, pylint, mypy
- PHP : phpstan, phpmd, phpcloc, psalm
- Transversal : SonarQube, Code Climate

Règle du boy scout (Robert C. Martin)

- Laisser le code dans un meilleur état qu'on ne l'a trouvé — renommer une variable, extraire une fonction, ajouter un test à chaque passage

Code smells avancés

*Feature Envy · Shotgun Surgery · Divergent Change · Primitive Obsession ·
et autres*

Feature Envy & Shotgun Surgery

Feature Envy

- Définition : une méthode plus intéressée par les données d'une autre classe que par les siennes
- Signal : enchaînement de getX(), getY(), getZ() sur le même objet externe
- Exemple : OrderPrinter qui appelle order.getCustomer(), order.getLines(), order.getDiscount() en série
- Correction : Move Method — déplacer la méthode dans la classe dont elle utilise les données
- Exception : les objets Strategy ou Visitor (délibérément externes)

Shotgun Surgery

- Définition : un seul changement conceptuel oblige à modifier de nombreux fichiers dispersés
- Signal : modifier une règle métier (ex. TVA à 20%) nécessite de retrouver et toucher 5 fichiers ou plus
- Lien avec SOLID : conséquence directe d'une violation du DIP (dépendances à des valeurs concrètes)
- Correction : Extract Class puis centraliser la règle en un seul endroit (Single Source of Truth)

Divergent Change & autres smells notables

Divergent Change

- Définition : une seule classe change pour des raisons multiples et non liées
- Signal : l'historique Git montre des commits provenant de contextes totalement différents
- Correction : Extract Class — séparer selon les acteurs qui demandent les changements
- Divergent Change vs Shotgun Surgery : l'un touche trop de classes, l'autre change pour trop de raisons

Primitive Obsession

- Définition : utiliser des types primitifs là où un objet valeur serait plus expressif et plus sûr
- Exemples : string pour un email (sans validation), float pour un montant (sans devise), int pour un identifiant
- Correction : Introduce Value Object — Email, Money, OrderId

Data Clumps

- Groupe de variables qui voyagent toujours ensemble → regrouper dans un objet dédié

Middle Man

- Classe qui ne fait que déléguer sans valeur ajoutée → supprimer l'intermédiaire ou lui donner une vraie responsabilité

Long Parameter List

- Au-delà de 3 paramètres → Introduce Parameter Object avec des champs nommés

Session 2

Refactoring d'une base de code existante

Catalogue Fowler · Méthode Mikado · Refactorer ou réécrire

Refactoring — Définition & principes (Martin Fowler, 1999)

Définition

- « Une technique disciplinée de restructuration du code existant qui consiste à modifier sa structure interne sans modifier son comportement observable. »

Ce que le refactoring EST

- Une activité de préparation : on assainit le code pour que la prochaine fonctionnalité soit plus facile à ajouter
- Une activité outillée : Extract Method, Rename, Move — disponibles dans tout IDE moderne
- Une activité sûre : les tests passent avant et après chaque étape

Ce que le refactoring N'EST PAS

- Corriger des bugs (même si des bugs peuvent apparaître en surface)
- Ajouter des fonctionnalités
- Réécrire le code from scratch
- Modifier sans couverture de tests — sans filet, c'est une réécriture risquée

Catalogue des refactorisations fondamentales

Extract Method

- Extraire un bloc identifiable en fonction nommée — déclencheur : un commentaire décrit ce que fait un bloc

Rename

- La refactorisation la plus impactante. Si une variable a un commentaire explicatif, le commentaire devient le nom

Move Method

- Déplacer une méthode dans la classe dont elle utilise le plus les données — corrige le Feature Envy

Inline Variable

- Supprimer une variable intermédiaire qui n'apporte pas de lisibilité

Introduce Parameter Object

- Regrouper les paramètres qui voyagent toujours ensemble dans une classe dédiée

Replace Conditional with Polymorphism

- Remplacer un switch/if-cascade sur un type par du polymorphisme (prépare l'OCP)

Disponibles nativement dans les IDE

- PHPStorm · IntelliJ · VS Code — raccourcis : Refactor > Extract Method / Rename / Move Class

Méthode Mikado — Refactoring complexe sans blocage (Elneby & Kniberg, 2010)

Principe

- Dans le jeu de Mikado, retirer un bâtonnet trop vite fait tomber les autres
- Avant de modifier, on identifie les prérequis — puis les prérequis des prérequis
- On commence par les feuilles (sans dépendances) pour remonter vers l'objectif
- À tout moment, le code est dans un état valide (les tests passent)

Bénéfice clé

- On n'est jamais dans un état intermédiaire bloquant
- Le revert systématique (git checkout -- .) est la règle, pas l'exception
- Chaque feuille validée réduit le graphe et rapproche de l'objectif

Algorithme en 5 étapes

1. Définir l'objectif (racine) : « Remplacer SQLite par PostgreSQL dans UserRepository »
2. Essayer de l'atteindre directement → compiler / exécuter les tests
3. Si les tests cassent : revert immédiat + ajouter un nœud enfant pour chaque prérequis découvert
4. Répéter depuis le bas du graphe (feuilles sans prérequis en premier)
5. Quand une feuille est atteinte sans casse : valider et monter d'un niveau

Refactorer ou réécrire ?

Arguments pour le refactoring progressif

- Préservation du savoir tacite : le code legacy encode des règles métier que personne ne sait reformuler (Spolsky, 2000)
- Continuité de service : livraison de fonctionnalités sans gel du projet
- Risque maîtrisé : chaque étape couverte par les tests existants
- Règle : préférer toujours le refactoring progressif

Arguments pour la réécriture

- Le code est tellement couplé qu'aucun test n'est possible sans infrastructure complète
- Les exigences non fonctionnelles ont radicalement changé (monolithe → système distribué temps réel)
- Les fondations technologiques sont obsolètes et non maintenables (framework abandonné, EOL sans sécurité)
- Condition sine qua non : disposer de tests d'intégration réutilisables pour valider la réécriture

Résumé

Concept	Ce qu'il résout	Outil associé
Nommage expressif	Code illisible, commentaires inutiles	Convention d'équipe, revue de code
Abstraction par niveau	Fonctions mélangées haut/bas niveau	Extract Method, SRP
Loi de Déméter	Couplage accidentel entre objets	Tell, don't ask
POLA	Comportements surprenants	Conception des API, revue
Dette technique	Raccourcis non documentés qui s'accumulent	Quadrant Fowler, ADR, règle du boy scout
Feature Envy	Méthode mal placée	Move Method
Shotgun Surgery	Règle dispersée dans plusieurs fichiers	Extract Class, DIP
Divergent Change	Classe qui change pour trop de raisons	Extract Class, SRP
Méthode Mikado	Refactoring complexe sans état intermédiaire bloquant	Graphe de prérequis, revert systématique